

Hotel Transilvania

Implicancias de implementar el atributo nombre en Cliente

Documento técnico de análisis de impacto. Describe el estado actual del modelo Cliente, los cambios necesarios por capa, lo que no se modifica, riesgos, decisiones de diseño y recomendación final.

Microservicio afectado principalmente: ms-cliente-service (puerto 8082)

Base de datos: clientes_db | Tabla: cliente

1. Situación actual

La entidad Cliente no posee un campo nombre de persona. Se identifica mediante rutDocumento (único), teléfono, dirección, rolCliente y tipoCliente.

Campos actuales (Cliente.java / V1__crear_tabla_cliente.sql):

- idCliente (PK)
- rutDocumento (UNIQUE, obligatorio)
- telefono (obligatorio)
- direccion (obligatorio)
- rolCliente (obligatorio)
- tipoCliente (obligatorio)

Nota: el microservicio Usuario ya tiene el campo nombre, pero cumple otra función: es el nombre de usuario para login (admin, user), no el nombre del huésped.

2. Por qué evaluar este cambio

- Dominio hotelero: un huésped se reconoce por nombre y documento.
- Mejor experiencia en listados, notificaciones e informes.
- El impacto técnico es acotado: otros MS solo referencian idCliente.
- No es obligatorio para que reservas o pagos funcionen hoy.

3. Cambios obligatorios (ms-cliente-service)

3.1 Modelo y persistencia

- Cliente.java: nuevo atributo nombre con @Column y validación (@NotBlank).
- Nueva migración Flyway V2__agregar_nombre_cliente.sql (ALTER TABLE). No modificar V1 si ya se ejecutó en algún entorno.
- ClienteService.actualizar(): agregar setNombre(...).
- ClienteService.guardar(): opcional log con nombre.

ClienteController: sin cambio de rutas; el JSON de POST/PUT incluye nombre.

3.2 Ejemplo de migración SQL

```
-- V2__agregar_nombre_cliente.sql (ejemplo)
ALTER TABLE cliente
ADD COLUMN nombre VARCHAR(100) NOT NULL DEFAULT "";
```

-- Luego actualizar registros existentes y quitar DEFAULT si se desea.

3.3 Ejemplo de body API (Postman)

POST http://localhost:8082/api/v1/clientes

Authorization: Bearer <token_admin>

```
{
  "rutDocumento": "11111111-1",
  "nombre": "Juan Pérez",
  "telefono": "+56911111111",
  "direccion": "Calle Principal 100",
  "rolCliente": "HUESPED",
  "tipoCliente": "NACIONAL"
}
```

4. Cambios recomendados (integraciones)

Otros microservicios consumen GET /api/v1/clientes/{id} vía WebClient y deserializan la respuesta en ClienteDTO. Conviene agregar el campo nombre en los DTO para reflejar el JSON (Jackson lo mapea sin cambiar la lógica).

Microservicio	Archivo	Cambio
ms-reserva-service	dto/ClienteDTO.java	Agregar private String nombre
ms-resena-service	dto/ClienteDTO.java	Agregar nombre (hoy solo: id, rut, tipo)
ms-notificacion-service	dto/ClienteDTO.java	Agregar nombre

ClienteClient en reserva, reseña y notificación: sin cambio de código. Siguen llamando obtenerClientePorId(id).

5. Lo que NO cambia

- Reserva, Reseña, Notificación, Pago: solo guardan idCliente (referencia lógica).
- Migraciones Flyway de otros microservicios.
- Endpoints de otros MS (solo validan existencia del cliente por ID).
- Usuario (auth): modelo independiente; ya tiene nombre de login.

6. Flujo de validación actual (sin usar campos del DTO)

ReservaService.validarReservaCompleta() llama clienteClient.obtenerClientePorId(). Si el cliente no existe devuelve 404 o excepción. No lee rut ni teléfono del DTO. Agregar nombre no altera este flujo salvo que se quiera validar el campo después.

7. Base de datos y datos existentes

Si ya hay filas en la tabla cliente, definir estrategia:

- Opción A: columna NOT NULL con DEFAULT "" y luego UPDATE manual.
- Opción B: columna nullable, rellenar datos, luego NOT NULL en V3.
- Bases nuevas: pueden incluir nombre en V1 solo si Flyway nunca corrió.

Scripts seed (scripts/sql/02_clientes_db.sql): agregar columna nombre en INSERT.

8. Decisiones de diseño a acordar

Tema	Opciones
Obligatoriedad	Recomendado: @NotBlank + NOT NULL
Unicidad	No necesario; rutDocumento ya es UNIQUE
Longitud	VARCHAR(100) o VARCHAR(255)
Búsqueda	Opcional: findByNombre o GET ?nombre=

9. Riesgos y mitigación

- Flyway checksum: no editar V1; usar solo V2.
- Clientes sin nombre en BD: plan de migración de datos.
- Postman/colecciones desactualizadas: actualizar ejemplos JSON.
- DTOs desalineados: tres ClienteDTO distintos hoy; buen momento para alinearlos.

10. Mejoras opcionales posteriores

- Notificaciones personalizadas: usar cliente.getNombre() en el mensaje.
- Filtro GET /clientes?nombre=... en ClienteController.
- Documentación técnica y PDFs del proyecto.

11. Esfuerzo estimado

- Mínimo (cliente + V2 + service): 3-4 archivos, complejidad baja.
- Recomendado (+ 3 DTOs + seed SQL): 6-8 archivos, complejidad baja.
- Completo (+ búsqueda + docs): 10+ archivos, complejidad media.

12. Diagrama de impacto (texto)

```
ms-cliente-service <-- CAMBIO PRINCIPAL
  Cliente.java + V2 SQL + ClienteService
    |
    | GET /clientes/{id} (JSON incluye "nombre")
    v
+-----+-----+-----+
| ms-reserva | ms-resena | ms-notificacion |
| ClienteDTO | ClienteDTO | ClienteDTO      |
| +campo nombre | +campo nombre | +campo nombre |
| (logica igual) | (logica igual) | (logica igual) |
+-----+-----+-----+

reserva / resena / notificacion / pago --> idCliente --> SIN CAMBIO
```

13. Recomendación final

No es estrictamente necesario para el funcionamiento actual del sistema, porque las integraciones solo validan el id del cliente. Si el objetivo es un sistema hotelero realista, presentación al cliente

del encargo o mejor trazabilidad del huésped, se recomienda implementar nombre como campo obligatorio. El costo de desarrollo es bajo y el riesgo es manejable con una migración V2 y actualización de datos de prueba.

Comparación: Usuario ya tiene nombre (login). Cliente es el que carece de nombre de persona; son cambios independientes.